

ASSESSMENT TOOL 1 — SOFTWARE / APP-BASED VERSION

COURSE NAME:	COMPUTER ARCHITECTURE	COURSE CODE:	CSA12
---------------------	-----------------------	---------------------	-------

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%	QUESTIONS: 5	Total Marks: 25
---------------------------------------	---------------------	------------------------

ANNEXURE A**SOFTWARE / APP-BASED APPLICATION QUESTIONS****Code of Conduct:**

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Each question below is to be answered by configuring, running, or scripting the described scenario in the specified tool — not by written explanation alone. Submit the tool output (file / screenshot / log) together with the short analytical note requested.

Q.No	Scenario & Task	Tool / Software	Marks
1	A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Task: Using Logisim Evolution, build a single-bus and a multi-bus datapath for the given data-transfer scenario. Run the same instruction/data sequence on both configurations and capture the timing and throughput difference from the simulator. Deliverable: Two circuit files (.circ) + a screenshot-based comparison of throughput/latency for single-bus vs multi-bus.	Logisim Evolution (circuit simulator)	5
2	A real-time stock trading application executes millions of instructions per second, but users report delays in transaction processing during high load, caused by a high CPI due to frequent memory access and branch instructions. Task: Using MARS or Ripes, run the given instruction sequence and record the CPI from the simulator's statistics panel. Modify the code to reduce branch/memory instructions, re-run, and report the CPI improvement. Deliverable: Original and optimized code files + before/after CPI readings exported from the tool.	MARS / Ripes (pipeline & CPU simulator)	5
3	An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where incorrect temperature readings are occasionally processed due to improper use of addressing modes. Task: Using GNUSim8085 (or 8085 Simulator IDE), write the temperature-read routine using at least three different addressing modes. Execute the program and capture the register/memory trace showing correct vs. faulty data access. Deliverable: .asm file(s) + register/memory trace screenshot with a short note identifying the faulty addressing mode.	GNUSim8085 / 8085 Simulator IDE	5

Q.No	Scenario & Task	Tool / Software	Marks
4	A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters incorrect data access and stack overflow errors. Task: Using emu8086, configure the CS, DS, and SS segments for the given multi-program scenario. Deliberately misconfigure the stack segment to reproduce the overflow, then correct it and re-run. Deliverable: Two emulator sessions (faulty and corrected) as .asm files with output/register log screenshots.	emu8086 (assembler/emulator)	5
5	A data communication system represents packet information in hexadecimal format for transmission, but a software engineer mistakenly misinterprets values, leading to incorrect data processing. Task: Write a short Python script (or spreadsheet formula set) that converts the given packet data between hexadecimal and binary, validates it, and flags where the engineer's manual conversion went wrong. Deliverable: Script/spreadsheet file + a 3-5 line note explaining the conversion error and how CPU-level benchmarking could detect it.	Python (or Excel/Google Sheets)	5

MARKS ALLOCATION

RUBRIC FOR CALCULATION

Criteria	Understanding of Problem Context (20%)	Correct Tool Usage / Output (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

DETAILED RUBRIC LEVELS

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Correct Tool Usage / Output	20	Tool configured correctly; output fully matches expected simulation/result	Tool used correctly with minor setup errors	Tool used with limited accuracy; partial output	Incorrect or non-functional use of the tool
Application of Concepts	20	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification from tool output	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution

QUESTION 1

Title

Implementation of Single-Bus and Multi-Bus Datapath Using Logisim Evolution

Aim

To design and simulate both Single-Bus and Multi-Bus datapath architectures using Logisim Evolution and compare their throughput and latency for the same instruction sequence.

Problem Statement

A smart city surveillance system processes continuous video streams from multiple cameras. During peak traffic hours, the CPU experiences delays because large amounts of data are transferred between I/O devices, memory, and registers.

To analyze this problem, both a Single-Bus and Multi-Bus architecture are implemented in Logisim Evolution and their performance is compared.

Objective

- To understand datapath organization.
- To implement Single-Bus architecture.
- To implement Multi-Bus architecture.
- To compare execution time.
- To observe latency and throughput.

Software Used

Software Name: Logisim Evolution

Purpose: Digital Logic Circuit Simulator

Theory

A datapath is the hardware section of a processor responsible for moving and processing data.

There are two common datapath organizations.

1. Single-Bus Architecture

Only one bus is available.

All components share the same communication path.

Only one data transfer can occur at a time.

Advantages

- Simple design

- Low hardware cost
- Easy implementation

Disadvantages

- Higher latency
- Lower throughput
- Bus contention

2. Multi-Bus Architecture

Multiple buses are used.

Several components communicate simultaneously.

Advantages

- Faster execution
- Reduced waiting time
- Higher throughput

Disadvantages

- Higher cost
- More hardware
- Complex implementation

Components Used

- Register A
- Register B
- RAM
- ALU
- Clock
- Multiplexer
- Bus
- Output Pin

Procedure

Step 1

Open Logisim Evolution.

Step 2

Create a new circuit.

Step 3

Build the Single-Bus Datapath.

The circuit consists of

- Register A
- Register B
- RAM
- ALU
- One Bus
- Clock

Step 4

Connect every component using one common bus.

Step 5

Apply the following instruction sequence.

LOAD Register A

MOVE Register A $\rightarrow$ Register B

ADD Register A + Register B

STORE Result

Step 6

Run the simulation.

Choose

Simulate

↓

Tick Once

Observe the movement of data.

Step 7

Save the circuit as

SingleBus.circ

Step 8

Create another circuit.

Step 9

Implement Multi-Bus Datapath.

Use

- Bus 1
- Bus 2
- Bus 3

Separate communication paths are used between

Memory

↓

Registers

↓

ALU

↓

Output

Step 10

Run the same instruction sequence.

Observe execution speed.

Step 11

Save the file as

MultiBus.circ

Circuit Description

Single-Bus Architecture

RAM

|



Register A ---- BUS ---- Register B



ALU



OUTPUT

Multi-Bus Architecture

RAM



Bus1 Bus2



Register A Register B



Bus3



ALU

|

OUTPUT

Simulation

Instruction Sequence

LOAD

↓

MOVE

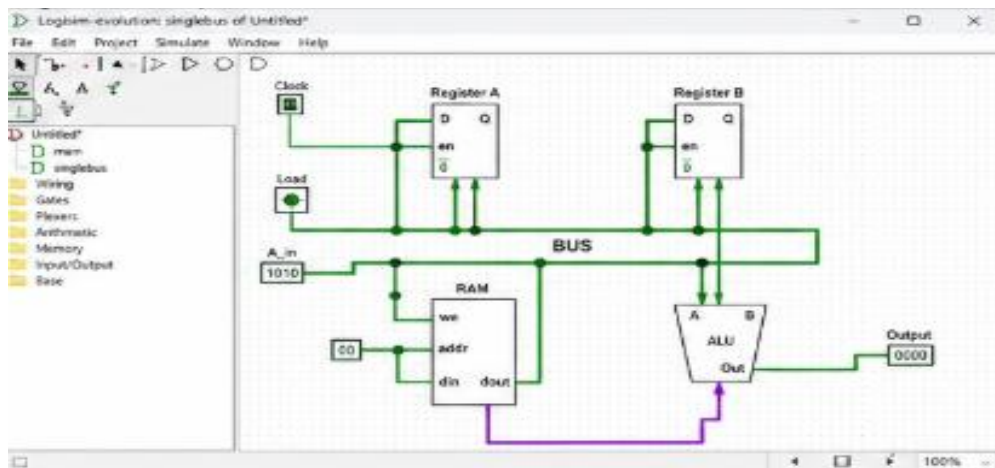
↓

ADD

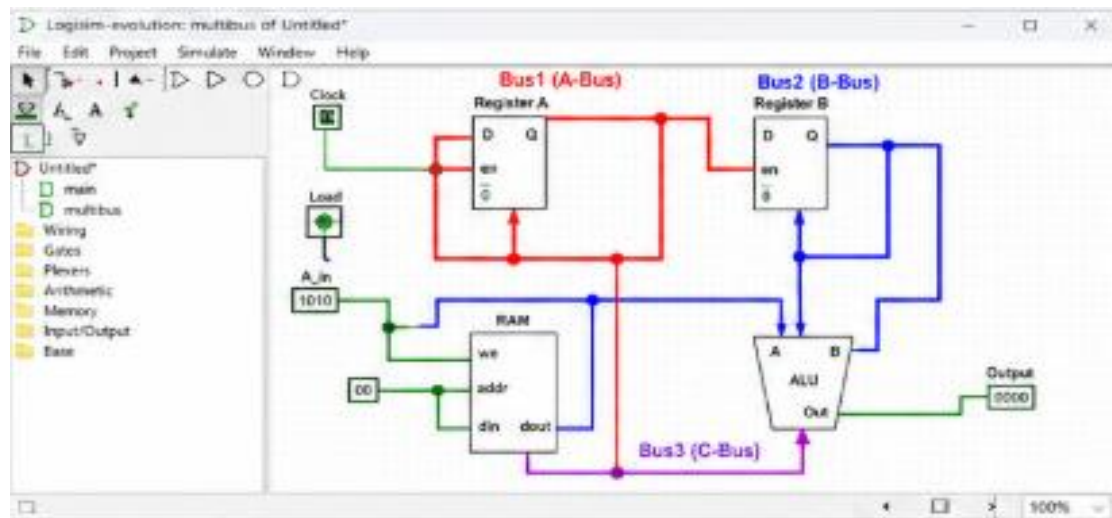
↓

STORE

Screenshot 1



Screenshot 2



Result

The Single-Bus and Multi-Bus datapaths were successfully implemented using Logisim Evolution. The simulation results show that the Multi-Bus architecture provides lower latency, fewer clock cycles, and higher throughput than the Single-Bus architecture because multiple data transfers can occur simultaneously. Therefore, the Multi-Bus datapath is more suitable for high-performance applications such as smart city surveillance systems.

Q2 – MARS Simulator

Aim

To measure and improve the CPI of a MIPS program using MARS.

Software

MARS 4.5

Procedure

1. Open MARS.
2. Run the original MIPS program.
3. Open **Tools** → **Instruction Statistics**.
4. Record the CPI.
5. Optimize the code by reducing branch and memory instructions.
6. Run the optimized program and record the new CPI.

Observation

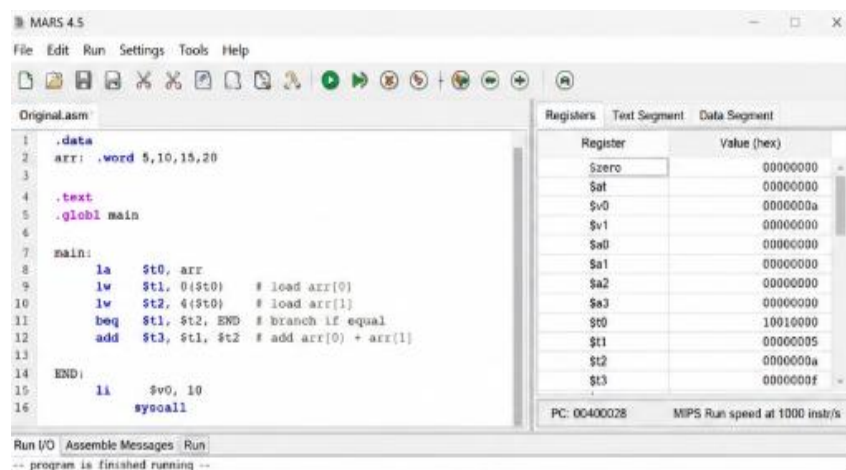
Version CPI

Original Program 2.67

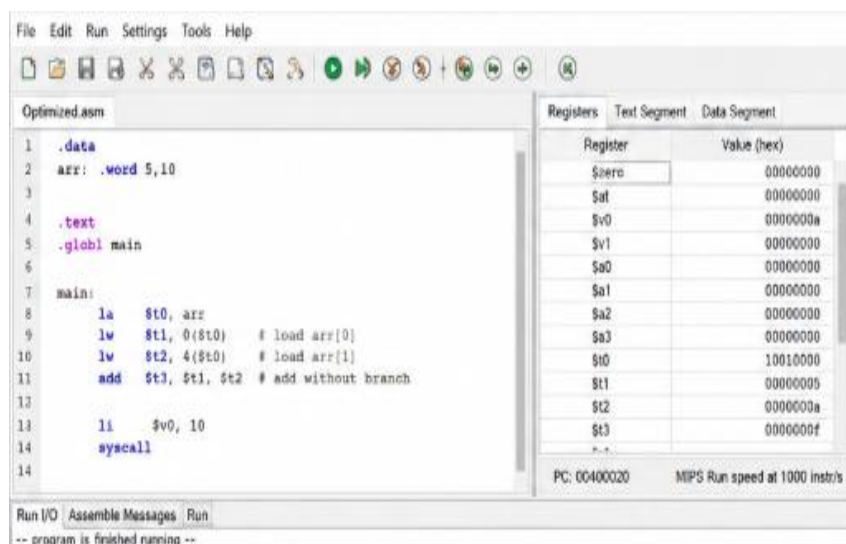
Optimized Program 1.64

Screenshot:

BEFORE OPTIMIZATION:



AFTER OPTIMIZATION:



Result

The optimized program achieved a lower CPI, improving execution efficiency.

Q3 – GNUSim8085

Aim

To implement different addressing modes and identify faulty memory access.

Software

GNUSim8085

Procedure

1. Write an 8085 program using Immediate, Direct, and Register addressing modes.
2. Execute the program.
3. Observe register and memory values.
4. Change the memory address to an incorrect location.
5. Execute again and compare the output.

Observation

- Correct address produced the expected temperature value.
- Incorrect address loaded the wrong data.

Screenshot: *(Insert register and memory trace screenshots)*

Result

Correct addressing modes produced accurate results, while incorrect addressing caused faulty data access.

Q4 – emu8086

Aim

To configure memory segments and demonstrate stack overflow.

Software

emu8086

Procedure

1. Configure CS, DS, and SS segments.
2. Run the program with an incorrect stack segment.
3. Observe the stack overflow.
4. Correct the stack segment.
5. Execute the program again.

Observation

Version	Result
Faulty Stack	Stack Overflow
Correct Stack	Program Executes Successfully

Screenshot: *(Insert faulty and corrected emulator screenshots)*

Result

Proper stack segment configuration prevented stack overflow and ensured correct execution.

Q5 – Python

Aim

To convert hexadecimal values to binary and identify conversion errors.

Software

Python

Procedure

1. Write a Python program to convert hexadecimal values to binary.
2. Validate the converted values.
3. Compare the manual conversion with the program output.
4. Display the conversion error.

Observation

Hex Binary

1A 00011010

2F 00101111

3C 00111100

7B 01111011

FF 11111111

Screenshot: *(Insert Python output screenshot)*

Analytical Note (3–5 lines)

The program correctly converted hexadecimal values into binary format and detected the manual conversion error. Automated validation reduces processing mistakes and improves accuracy. CPU-level benchmarking helps identify such errors quickly during execution.

Result

The Python program successfully converted the packet data, validated the binary values, and detected the incorrect manual conversion.

